

# Project Overview

This project is a Mobile Automation Testing Framework developed using:

- Java
- Appium (Latest Version)
- Selenium WebDriver
- Cucumber BDD
- TestNG
- Maven

The framework automates the Android application under test using Appium and follows industry-standard automation framework design practices.

---

## Framework Features

### 1. BDD Framework using Cucumber

Feature files are implemented using Gherkin syntax.

Example:

Feature: Login Functionality

Scenario: Successful login

Given user is on login screen

When user enters username "Raksha"

And user enters password "Password123"

And user enters dob "05101995"

And user taps submit

Then search results should be displayed

---

### 2. Page Object Model (POM)

The framework follows the Page Object Model design pattern.

Benefits:

- Reusable page methods
- Better maintainability
- Reduced code duplication
- Easy scalability

Example:

- BasePage.java
  - LoginPage.java
- 

### 3. Reusable Framework Architecture

The framework is designed to support automation for multiple Android applications with minimal changes.

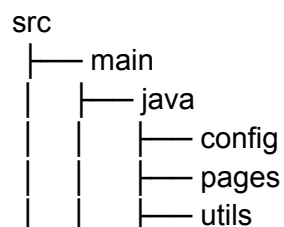
Reusable Components:

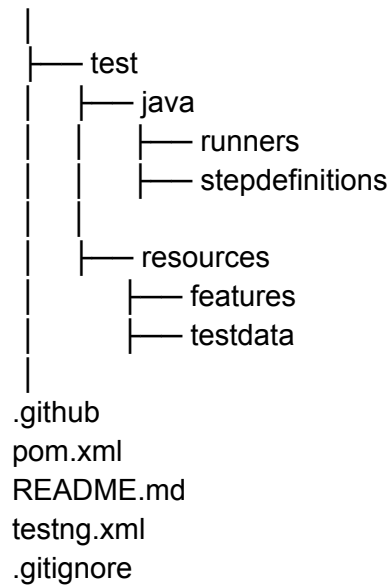
- DriverManager
- ConfigReader
- BasePage
- Hooks
- RetryAnalyzer
- Utilities

Only the following need updates for a new app:

- App capabilities
  - App package/activity
  - Locators
  - Test data
  - Feature files
- 

## Project Structure





---

## Supported Execution Platforms

The framework supports execution on:

- Android Emulator
- Real Android Devices
- Device Cloud Platforms
  - BrowserStack
  - SauceLabs
  - LambdaTest

---

## Appium Setup Details

### Installed Components

- Node.js
- Appium 3+
- UiAutomator2 Driver
- Android Studio
- Android SDK
- Java 17

- Maven
- 

## Device Validation Commands

### Verify Connected Device

adb devices

### Verify App Package & Activity

adb shell dumpsys window | grep -E 'mCurrentFocus'

---

## Framework Configuration

### Capabilities Used

```
{
  "platformName": "Android",
  "automationName": "UiAutomator2",
  "deviceName": "Medium_Phone_API_34.1",
  "udid": "emulator-5554",
  "platformVersion": "14",
  "appPackage": "com.tui.qa.challenge",
  "appActivity": "com.tui.qa.challenge.MainActivity",
  "noReset": false,
  "adbExecTimeout": 120000,
  "dontStopAppOnReset": true,
  "newCommandTimeout": 300
}
```

---

## Test Data Management

The framework uses external JSON test data.

Example:

```
{
  "validUser": {
    "username": "Raksha",
    "password": "Password123",
    "dob": "05051994"
  }
}
```

Benefits:

- Easy maintenance
  - Separation of test data from code
  - Better reusability
- 

## Reporting

The framework generates:

- Cucumber HTML Reports
- JSON Reports

Report Location:

target/cucumber-report.html

---

## Retry Mechanism

RetryAnalyzer is implemented to rerun failed tests automatically.

Benefits:

- Reduces flaky failures
- Improves execution stability
- Useful for transient Appium issues

---

# Assertions

Assertions are implemented using TestNG.

Example:

```
Assert.assertTrue(true);
```

Framework can be extended with business validations.

---

# Locator Strategy

Preferred locator hierarchy:

1. accessibilityId
2. resource-id
3. androidUIAutomator
4. xpath

This improves:

- Execution speed
  - Stability
  - Maintainability
- 

# Framework Stability Improvements

The following stability measures were implemented:

- Explicit waits
- Retry mechanism
- Reusable driver management
- Clean Appium session handling
- Optimized locators

- JSON-based test data
- 

## CI Pipeline

A basic non-functional GitHub Actions CI pipeline is included.

Location:

`.github/workflows/mobile-tests.yml`

Pipeline performs:

- Project checkout
  - Java setup
  - Maven build execution
- 

## Prerequisites & Dependencies

Before running the framework, ensure the following tools are installed:

### Required Software

| Software       | Version          |
|----------------|------------------|
| Java           | 17               |
| Maven          | Latest<br>Stable |
| Node.js        | Latest LTS       |
| Appium         | 3+               |
| Android Studio | Latest<br>Stable |
| Android SDK    | Installed        |

Appium UiAutomator2 Driver    Installed

---

## Dependency Installation

### Install Node.js

Download and install:

<https://nodejs.org/>

---

### Install Appium

`npm install -g appium`

Verify installation:

`appium -v`

---

### Install UiAutomator2 Driver

`appium driver install uiautomator2`

Verify installed drivers:

`appium driver list --installed`

---

### Install Android Studio

Download:

<https://developer.android.com/studio>

Ensure the following are installed:

- Android SDK
  - Android SDK Platform Tools
  - Android Emulator
  - Android API 33 or API 34 Emulator
- 

## Environment Variables

Configure:

```
JAVA_HOME  
ANDROID_HOME  
PATH
```

Example:

```
export JAVA_HOME=/Library/Java/JavaVirtualMachines/jdk-17.jdk/Contents/Home  
export ANDROID_HOME=$HOME/Library/Android/sdk  
export PATH=$PATH:$ANDROID_HOME/platform-tools
```

---

## Emulator Setup

### Start Android Emulator

```
emulator -avd YOUR_AVD_NAME
```

Verify emulator:

```
adb devices
```

Expected:

```
emulator-5554 device
```

---

# Steps to Run the Framework

## Step 1 – Start Emulator

`emulator -avd YOUR_AVD_NAME`

---

## Step 2 – Start Appium Server

`appium`

Expected:

Appium REST http interface listener started

---

## Step 3 – Verify Connected Device

`adb devices`

---

## Step 4 – Run Test Execution

`mvn clean test`

---

## Step 5 – View Reports

Open:

`target/cucumber-report.html`

in browser.

---

# Terminal Execution

## Start Appium

appium

## Run Tests

mvn clean test

---

# Cleanup & Code Quality

The framework has been cleaned for submission:

- Removed unused utilities
  - Removed dead code
  - Removed empty methods
  - Removed duplicate imports
  - Validated locators
  - Ensured end-to-end execution
- 

# Git Ignore

A `.gitignore` file is included to exclude:

- target/
  - IDE files
  - logs
  - screenshots
  - generated reports
- 

# End-to-End Flow Covered

Launch App  
→ Enter Username  
→ Enter Password  
→ Enter DOB  
→ Submit  
→ Assertion  
→ Driver Quit

---

## Deliverables

The following deliverables are included:

- Source Code
  - Feature Files
  - Test Data
  - README.md
  - Cucumber Reports
  - CI Pipeline
  - Git Repository
- 

## Repository Details

Code repository is shared via:

- GitHub
- 

## Conclusion

The automation framework is scalable, reusable, and follows standard automation testing best practices.

The framework is capable of supporting future mobile automation enhancements and can be integrated with cloud execution platforms and CI/CD pipelines.